

nanospice

A SPICE circuit simulator in 1000 lines of Rust

Milan Rother

2026

Abstract

nanospice is a circuit simulator with the classic Berkeley SPICE algorithm set: modified nodal analysis, Newton-Raphson with junction limiting and two operating point fallbacks, trapezoidal transient integration with local truncation error timestep control, and small-signal AC analysis. The entire solver lives in one Rust source file of at most 1000 nonblank, noncomment lines, enforced by a test, with no dependencies outside the standard library. This report derives each algorithm, explains the design decisions the budget forced, and maps both to the code. Each section header states how many of the 999 lines the corresponding code section spends.

1 The budget as a design tool

The rule: one file, `src/main.rs`, at most 1000 lines after removing blank lines and comments, standard library only. A test reads the file, counts, and fails the build above the limit. Tests, example netlists, and comments are free. The point of the rule is not minimalism for its own sake. A hard budget forces every feature to displace another, which makes the cost of each decision explicit, and it keeps the whole simulator readable in one sitting, which is the educational goal of the repository.

Table 1 shows where the lines go. The parser is the largest consumer, which is typical: the numerics of a circuit simulator are compact, the input format is not.

section	lines	cumulative
constants, error and output plumbing	23	23
numbers: <code>Num</code> trait, complex, unit suffixes	78	101
circuit data model	63	164
parser	262	426
sparse LU	96	522
device models	50	572
solver: assembly, Newton, operating point	179	751
output selection	31	782
analyses and main	217	999

Table 1: Line budget by code section, in file order.

Everything the budget excluded is listed with reasons in section 10.

2 Modified nodal analysis

solver, shared with sections 4 and 5: 179 lines, running total 751 of 999

The unknown vector stacks the node voltages and one branch current for every voltage-defined element:

$$x = (v_1, \dots, v_N, i_1, \dots, i_B)^T,$$

where the branches belong to voltage sources, inductors, and VCVS elements. Kirchhoff's current law gives one equation per node; each voltage-defined element adds one equation of the form $v_p - v_n = E$ and contributes its current $\pm i$ to the node equations. A two-terminal conductance g between nodes p and n stamps the familiar quad

$$\begin{pmatrix} +g & -g \\ -g & +g \end{pmatrix}$$

into rows and columns (p, n) . A VCCS is the same quad with the rows taken at the output nodes and the columns at the controlling nodes, so one helper covers both:

```
fn quad(&mut self, p: usize, n: usize, cp: usize, cn: usize, g: T) {
    self.add(p, cp, g);
    self.add(p, cn, T::zero() - g);
    self.add(n, cp, T::zero() - g);
    self.add(n, cn, g);
}
fn branch(&mut self, p: usize, n: usize, br: usize) {
    self.add(p, br, T::one());
    self.add(n, br, T::zero() - T::one());
    self.add(br, p, T::one());
    self.add(br, n, T::zero() - T::one());
}
```

The ground row. Ground is unknown index 0, and it is a dummy: `add` silently drops any entry with a row or column index of 0, and the solver loops start at index 1 with $x_0 = 0$ pinned. The alternative is the usual *if node is not ground* test inside every stamp, roughly two dozen branches across the device set. Spending one guard inside `add` instead of one per stamp is the kind of trade the budget rewards, and it also removes a class of copy-paste bugs.

3 Sparse LU

sparse LU: 96 lines, running total 522 of 999

Dense LU with partial pivoting is about 25 lines and was the first implementation. It is $O(n^3)$, which caps practical circuits at a few hundred nodes. The replacement is a genuinely sparse LU in 96 lines, including the generic data structure.

Each matrix row is a vector of (column,value) pairs kept sorted by column. The factorization maintains one invariant:

Before elimination step k , every row stored at index $i \geq k$ contains only columns $\geq k$.

The invariant holds initially because column 0 is never stored, and step k preserves it because eliminating column k from a row removes that entry. Its consequence is the whole design: if a row at index $i \geq k$ has an entry in column k at all, that entry is the *first* element of its vector. Pivot search reduces to scanning first elements, and the update of a row against the pivot row is a merge of two sorted sequences in which fill-in appears automatically as the union of the column sets:

```

// row a minus f times pivot row p, both sorted
fn merge<T: Num>(a: &[(usize, T)], p: &[(usize, T)], f: T) -> Vec<(usize, T)> {
    let mut o = Vec::with_capacity(a.len() + p.len());
    let (mut i, mut j) = (0, 0);
    while i < a.len() && j < p.len() {
        if a[i].0 == p[j].0 {
            o.push((a[i].0, a[i].1 - f * p[j].1));
            i += 1;
            j += 1;
        } else if a[i].0 < p[j].0 {
            o.push(a[i]);
            i += 1;
        } else {
            o.push((p[j].0, T::zero() - f * p[j].1));
            j += 1;
        }
    }
    o.extend_from_slice(&a[i..]);
    for &(c, v) in &p[j..] {
        o.push((c, T::zero() - f * v));
    }
    o
}

```

Pivoting is by largest magnitude among the candidate first entries, with rows swapped whole. MNA needs this: the constraint row of a voltage source has a structural zero on the diagonal, so factorization without row exchanges fails on essentially every netlist. If no candidate exists the trailing matrix has an empty column and the matrix is singular; the failing column index is returned and the error message names the node or branch, which turns the most common user error, a node with no DC path, into a self-explaining diagnostic.

What is missing, deliberately, is fill-reducing ordering. Real SPICE uses Markowitz ordering; here the elimination order is netlist order, and the worst case degrades toward dense cost. For tree-like and ladder-like circuits, which is what a 1000-line simulator will realistically see, the netlist order is close to optimal anyway: a 2000-node RC ladder solves its operating point plus an AC sweep in tens of milliseconds.

4 Newton-Raphson and the operating point

device models, plus the solver drivers: 50 lines, running total 572 of 999

Nonlinear devices enter the linear system through companion linearization. A device current $i(v)$ is replaced at the current iterate $v^{(k)}$ by

$$i \approx g v + I_{eq}, \quad g = \left. \frac{\partial i}{\partial v} \right|_{v^{(k)}}, \quad I_{eq} = i(v^{(k)}) - g v^{(k)},$$

that is, a conductance in parallel with a current source. Assembling and solving this linearized system is one Newton iteration. Convergence is declared when every unknown moves less than $\text{tol}_i = \tau_i + \text{RELTOL} \cdot \max(|x_i^{(k)}|, |x_i^{(k+1)}|)$ between iterates, with $\tau_i = \text{VNTOL} = 1 \mu\text{V}$ for node voltages and $\text{ABSTOL} = 1 \text{ pA}$ for branch currents.

Junction limiting. The diode current $i = I_S(e^{v/V_T} - 1)$ makes plain Newton diverge: an early iterate easily proposes $v = 5 \text{ V}$, and $e^{5/0.02585}$ overflows any linearization. The classic

fix is `pnjlim`, which damps the proposed junction voltage logarithmically once it passes the critical voltage

$$v_{crit} = V_T \ln \frac{V_T}{\sqrt{2} I_S},$$

the point of maximum curvature of the exponential characteristic:

```
fn pnjlim(vnew: f64, vold: f64, vt: f64, vcrit: f64) -> f64 {
  if vnew > vcrit && (vnew - vold).abs() > 2.0 * vt {
    if vold > 0.0 {
      let arg = 1.0 + (vnew - vold) / vt;
      if arg > 0.0 { vold + vt * arg.ln() } else { vcrit }
    } else {
      vt * (vnew / vt).ln()
    }
  } else {
    vnew
  }
}
```

Limiting must deny convergence. This is the one place where nanospice shipped a real bug worth documenting. With limiting alone, the following happens on a common-emitter amplifier: the voltage divider puts the base at 2.1 V while the emitter is at ground, `pnjlim` clamps the base-emitter voltage seen by the stamps to 0.11 V, the transistor therefore conducts nothing, the node voltages stop changing, and the delta-based convergence test reports success with the transistor completely off. The solution is the classic one: any iteration in which a limiter actually changed a junction voltage is not allowed to converge. The assembly reports a `limited` flag and the Newton loop requires it to be false. With that in place the same amplifier walks the junction up in logarithmic steps and lands on the correct bias in a dozen iterations.

Fallbacks. When plain Newton fails, two continuation methods run in order. Gmin stepping solves the circuit with a large conductance g_{min} across every junction and reduces it decade by decade from 10^{-2} to 10^{-12} S, warm-starting each solve from the last. Source stepping scales all source values by a factor ramped from 0.1 to 1.0; the zero-source circuit is trivially solvable and each step tracks the solution continuously. Gmin stepping handles floating-junction trouble, source stepping handles feedback and bistable topologies, and together they are the standard SPICE fallback chain:

```
// second fallback: source stepping, ramp all sources up with warm starts
reset(x, lim);
for k in 1..10 {
  let fac = k as f64 / 10.0;
  if newton(ckt, x, lim, &Mode::Dc { fac }, GMIN, ITL_OP).is_none() {
    die("operating point did not converge");
  }
}
```

5 Device models

device stamps, inside the solver section: 179 lines, running total 751 of 999

Diode. Shockley equation with emission coefficient n : $i = I_S(e^{v/nV_T} - 1)$, conductance $g_d = I_S e^{v/nV_T} / nV_T$, plus g_{min} in parallel. The junction voltage passes through `pnjlim` with per-device memory of the last limited value.

MOSFET, level 1. The square-law model with channel-length modulation:

$$i_D = \begin{cases} 0 & v_{GS} \leq V_{T0} \\ K_P (v_{ov} v_{DS} - \frac{1}{2} v_{DS}^2) (1 + \lambda v_{DS}) & v_{DS} < v_{ov} \\ \frac{1}{2} K_P v_{ov}^2 (1 + \lambda v_{DS}) & v_{DS} \geq v_{ov} \end{cases} \quad v_{ov} = v_{GS} - V_{T0}.$$

The model is evaluated in an *effective frame* that handles both the pmos polarity and the source-drain swap for $v_{DS} < 0$. With the sign $s = \pm 1$ for nmos/pmos and effective terminals (d_e, s_e) chosen so that $v_{DS}^e \geq 0$, the actual terminal current is $I = s i_D$ and the chain rule gives, for example,

$$\frac{\partial I}{\partial v_g} = s \cdot \frac{\partial i_D}{\partial v_{GS}^e} \cdot \frac{\partial v_{GS}^e}{\partial v_g} = s \cdot g_m \cdot s = g_m.$$

The two signs cancel in every derivative, so the conductance stamps are identical for all four cases and only the right-hand side carries s . That one observation removes what is otherwise a four-way case split:

```
fn mos_op(dv: &Dev, x: &[f64]) -> (usize, usize, f64, f64, f64) {
  let Dev::M { d, g, s, kp, vt0, lambda, pmos } = dv else { unreachable!() };
  let sg = if *pmos { -1.0 } else { 1.0 };
  let (ed, es) = if sg * (x[*d] - x[*s]) >= 0.0 { (*d, *s) } else { (*s, *d) };
  let vgs = sg * (x[*g] - x[es]);
  let vds = sg * (x[ed] - x[es]);
  let (id, gm, gds) = mos_eval(vgs, vds, *kp, *vt0, *lambda);
  (ed, es, sg * id, gm, gds)
}
```

BJT. The Ebers-Moll model in transport form. With $e_f = e^{v_{BE}/V_T}$ and $e_r = e^{v_{BC}/V_T}$:

$$i_{CT} = I_S(e_f - e_r), \quad i_{BE} = \frac{I_S}{\beta_F}(e_f - 1), \quad i_{BC} = \frac{I_S}{\beta_R}(e_r - 1),$$

$$I_C = i_{CT} - i_{BC}, \quad I_B = i_{BE} + i_{BC}, \quad I_E = -(I_C + I_B).$$

Four small-signal conductances $g_{mf} = \partial i_{CT} / \partial v_{BE}$, $g_{mr} = -\partial i_{CT} / \partial v_{BC}$, $g_\pi = \partial i_{BE} / \partial v_{BE}$, $g_\mu = \partial i_{BC} / \partial v_{BC}$ fill the 3×3 Jacobian stamp

$$\begin{pmatrix} \partial I_C / \partial v_b & \partial I_C / \partial v_c & \partial I_C / \partial v_e \\ \partial I_B / \partial v_b & \partial I_B / \partial v_c & \partial I_B / \partial v_e \\ \partial I_E / \partial v_b & \partial I_E / \partial v_c & \partial I_E / \partial v_e \end{pmatrix} = \begin{pmatrix} g_{mf} - g_{mr} - g_\mu & g_{mr} + g_\mu & -g_{mf} \\ g_\pi + g_\mu & -g_\mu & -g_\pi \\ g_{mr} - g_{mf} - g_\pi & -g_{mr} & g_{mf} + g_\pi \end{pmatrix},$$

whose rows and columns each sum to zero, a quick correctness check for any MNA stamp. The pnp uses the same npn-frame equations through the sign trick above; both junctions carry their own `pnjlim` memory, which is why the limiter state is two values per device. An integration test builds the same common-emitter stage as npn and as pnp mirror image and requires the solutions to be exactly symmetric.

6 Transient analysis

analyses, with output selection: 217 lines, running total 999 of 999

Companion models. The trapezoidal rule applied to the capacitor equation $i = C dv/dt$ over a step h gives

$$\frac{i_{n+1} + i_n}{2} = C \frac{v_{n+1} - v_n}{h} \implies i_{n+1} = \underbrace{\frac{2C}{h}}_{g_{eq}} v_{n+1} - \underbrace{\left(\frac{2C}{h} v_n + i_n\right)}_{-I_{eq}},$$

a conductance g_{eq} in parallel with a history current source: the capacitor becomes a resistor for the duration of one step. The inductor is dual, $v_{n+1} = (2L/h)(i_{n+1} - i_n) - v_n$, stamped into its branch row. Each reactive device keeps a two-entry state, its value and the dual quantity, updated after every accepted step. The trapezoidal rule is A-stable and undamped; an integration test runs an LC tank for twenty periods and requires the amplitude to survive.

Timestep control. The local truncation error of the trapezoidal rule is $\varepsilon_{tr} = -\frac{h^3}{12} \ddot{x}(\xi)$. To estimate $\ddot{x}$ without extra solves, the integrator carries a quadratic polynomial predictor through the last three accepted points. With step history d_1, d_2 and the new step h , Lagrange extrapolation gives coefficients

$$\ell_0 = \frac{(h + d_1)(h + d_1 + d_2)}{d_1(d_1 + d_2)}, \quad \ell_1 = -\frac{h(h + d_1 + d_2)}{d_1 d_2}, \quad \ell_2 = \frac{h(h + d_1)}{(d_1 + d_2) d_2},$$

and the predictor error is $\varepsilon_p = \frac{1}{6} \ddot{x} h(h + d_1)(h + d_1 + d_2)$. Corrector and predictor bracket the same third derivative, so their difference measures it (a Milne-style estimate), and the trapezoidal share of the gap is

$$\varepsilon_{tr} \approx \frac{h^3/12}{h(h + d_1)(h + d_1 + d_2)/6 + h^3/12} (x_c - x_p),$$

which for uniform steps reduces to the classic factor $1/13$. The error is compared per unknown against $\text{TRTOL} \cdot \text{tol}_i$ with $\text{TRTOL} = 7$, the step is rejected above 1, and the next step follows the cube-root rule $h' = 0.9 h r^{-1/3}$, clamped to $[0.3, 2] \times h$ and capped at `tstep`:

```
let etrap = hs * hs * hs / 12.0;
let fac = etrap / (hs * (hs + d1) * (hs + d1 + d2) / 6.0 + etrap);
let mut r: f64 = 0.0;
for i in 1..m {
    r = r.max(fac * (xn[i] - xp[i]).abs() / (TRTOL * tolv(i, nn, xn[i], x[i])));
}
let scale = 0.9 / r.max(1e-8).cbrt();
```

The predictor doubles as the Newton starting point, which is free and usually saves an iteration. Before three points of history exist, and after a nonconvergent step (cut by 8, the SPICE2 reflex), the iteration count controls the step instead.

Breakpoints. A pulse edge in the middle of a step is invisible to the method: the companion history current at the interval start belongs to the pre-edge circuit, and the predictor extrapolates smoothness that is not there. The first RC test against the analytic step response failed at exactly the first output point for this reason. The classic remedy is exact: every corner of every pulse source (delay, rise end, fall start, fall end, per period) is registered as a breakpoint, steps are clamped to land on breakpoints exactly, and after crossing one the integrator restarts with a small step and cleared predictor history, since the waveform derivative is discontinuous there.

7 Small-signal AC

numbers: Num trait and complex: 78 lines, running total 101 of 999

AC analysis linearizes the circuit at the operating point and solves $(G + j\omega C)\tilde{x} = b$ per frequency, where b holds the ac source values. The implementation leans on one observation: at a converged operating point the limiters pass voltages through unchanged, so *the resistive matrix G is exactly the DC Jacobian that the last Newton iteration already assembles*. The AC path therefore reuses `assemble` verbatim, maps the result to complex, and adds only the reactive stamps $j\omega C$ and the branch entry $-j\omega L$. No device model contains AC-specific code.

The complex solve is the same 96-line sparse LU. The solver is generic over a three-method trait, implemented for `f64` and a 15-line complex type:

```
trait Num:
  Copy
  + std::ops::Add<Output = Self>
  + std::ops::Sub<Output = Self>
  + std::ops::Mul<Output = Self>
  + std::ops::Div<Output = Self>
{
  fn zero() -> Self;
  fn one() -> Self;
  fn mag(self) -> f64;
}
```

`mag` is the pivoting magnitude, absolute value for reals and modulus for complex. This trait plus operator implementations is the entire price Rust charges for not shipping a complex type in std; the alternative, two copies of the factorization, costs more lines and can drift apart.

8 The parser

parser: 262 lines, running total 426 of 999

The parser is the largest section and the least glamorous. Line discipline follows SPICE: the first line is a title, `*` comments, `;` trails, `+` continues, everything is case-insensitive, and parentheses and commas are replaced by whitespace once during preprocessing, which lets `sin(0 1 1k)` and `sin 0 1 1k` tokenize identically.

Numbers use the SPICE grammar: the longest prefix that parses as a float, then a unit suffix. The only subtlety is that `meg` must match before `m`, handled by table order:

```
fn num(tok: &str) -> Option<f64> {
  let t = tok.trim();
  if !t.is_ascii() {
    return None;
  }
  let mut end = t.len();
  while end > 0 && t[..end].parse::<f64>().is_err() {
    end -= 1;
  }
  if end == 0 {
    return None;
  }
  let v: f64 = t[..end].parse().unwrap();
```

```

    let mult = SUFFIXES.iter().find(|(s, _)| t[end..].starts_with(s)).map_or(1.0,
    |(_, m)| *m);
    Some(v * mult)
}

```

The ASCII guard exists because byte-indexed slicing panics inside multibyte characters; a netlist containing a literal micro sign must produce *bad number*, not a crash, and a fuzz test holds the parser to that.

`.model` cards are collected in a prepass so they may appear anywhere in the deck. A device line referencing a model has the model’s parameter tokens spliced in *behind* its inline tokens; since parameter lookup returns the first match, instance parameters override model parameters by construction, with no explicit precedence logic.

Subcircuits are the notable omission. `.subckt` costs roughly a hundred lines of name spacing and flattening and is the single feature most requested of any SPICE clone; it lost to the transistor models.

9 Validation

The test suite is 18 integration tests running the built binary on real netlists. Tests are free of the budget, so robustness lives there, and the suite leans on references that are independent of the implementation:

- Analytic: RC and RL step responses against $1 - e^{-t/\tau}$, the RC corner at $1/\sqrt{2}$ and -45° , LC amplitude $i_0\sqrt{L/C}$, and energy conservation over twenty periods.
- Bias points: MOSFET saturation current, BJT $I_C = \beta_F I_B$ consistency, diode KCL closure, and exact npn/pnp mirror symmetry.
- Structural: a 200-node ladder and a 30-stage LCG-randomized resistor ladder verified node by node against a Thevenin reduction computed independently in the test.
- Negative: seven malformed netlists (unknown devices, bad numbers, missing tokens, non-ASCII input, floating nodes) must exit with a clean named error and must not panic; the floating-node case must name the node.
- Meta: the LOC counter itself.

The LTE controller has a dedicated regression: an RC circuit whose time constant is ten times smaller than `tstep`, which a fixed-step integrator fails badly and the controller must resolve on its own.

10 The cut list

On language: the device enum with exhaustive matching carries the parser, the assembly, and the output selection, and is the main reason the budget holds. C++ would trade the roughly 60 lines the `Num` trait and complex type cost against visitor or inheritance boilerplate around `std::variant`, roughly a wash with fewer static guarantees. Fortran would shine on dense arrays, which this simulator does not use, and struggle badly on the parser and the sparse row structure.

feature	why it lost	est. lines
<code>.subckt</code> hierarchy	lost to the transistor models	~100
Markowitz ordering	netlist order suffices at this scale	~80
Gummel-Poon BJT	Ebers-Moll covers the teaching core	~80
junction capacitances	needs charge state on every junction	~60
noise analysis	needs the adjoint system	~80
<code>.param</code> expressions	a calculator is its own project	~60
current-based convergence check	limiting denial covers the failure mode	~15

Table 2: Features that did not fit, with the reason and the estimated cost.

11 Closing

The final count is 999 of 1000 lines, with the classic algorithm set complete: MNA, sparse LU with partial pivoting, Newton with `pnjlim` and both continuation fallbacks, trapezoidal integration with LTE control and breakpoints, small-signal AC, nine device types, four analyses, model cards, initial conditions, and output selection. The one line of headroom is left as a reminder that the next feature must pay for itself.